UT6 · PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

La Forja del Servidor Central

Antonio Cano Ruiz y Álvaro José Hernández Sánchez · 2.º de Desarrollo de Aplicaciones Multiplataforma

El reto

Una app Android que sintoniza vídeo remoto por puestos y avisa con campanas cuando termina la transmisión.

3

puestos

Minas Tirith, Isengard y Rivendell

2

reproductores

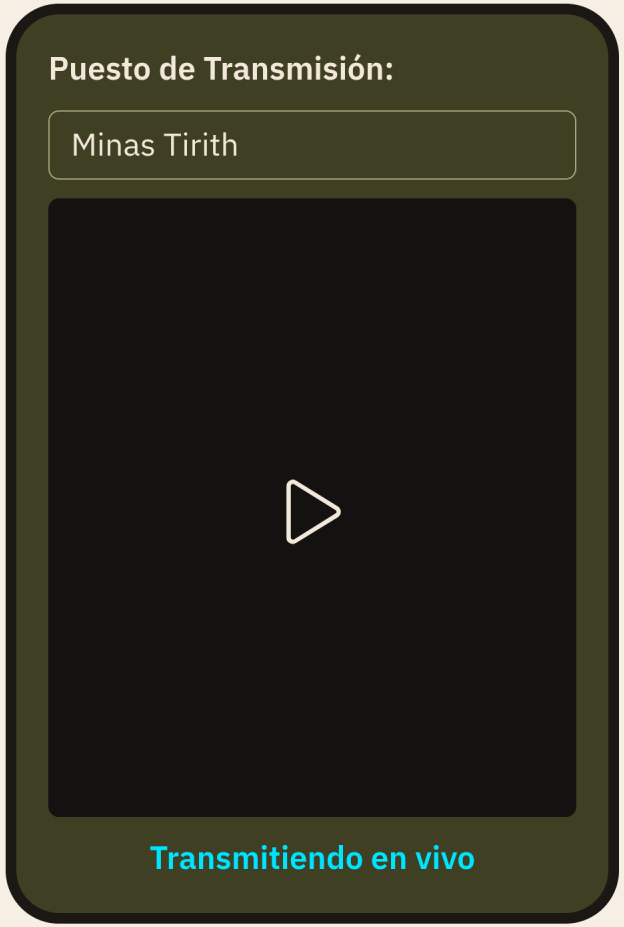
ExoPlayer para vídeo, MediaPlayer para alarma

4

estados de conexión

Inactiva, búfer, en vivo y terminada

La interfaz



Spinner
Lista de puestos, rellenada con un ArrayAdapter.

PlayerView
Muestra el vídeo y ocupa el espacio libre con layout_weight. Sus controles se ocultan a los 5 s.

TextView de estado
Traduce el estado del reproductor a un mensaje para el usuario.

Esquema de la pantalla, no captura real

La preparación

Permiso

`android.permission.INTERNET`

Declarado en el manifest para acceder a los vídeos remotos.

Media3

`media3-exoplayer, -ui y -common · 1.4.1`

Motor de reproducción, vista del vídeo y clases comunes.

Audio local

`res/raw/campanas_alarma.mp3`

Incluido en la app, así la alarma no depende de la red.

El flujo de la app



El Spinner solo inicia el cambio; después, el reproductor marca el ritmo.

Los dos reproductores

	ExoPlayer	MediaPlayer
Función	Vídeo de los puestos	Campanas de alarma
Origen	URL remota	Recurso en res/raw
Biblioteca	Media3 (dependencia)	API de Android
Creación	Al abrir la pantalla	Cada vez que acaba un vídeo
Liberación	En onDestroy	Al terminar el audio u onDestroy

Dos instancias independientes, cada una con su propia gestión de recursos.

Sintonizar un canal

```
private fun sintonizarCanal(url: String) {
    exoPlayer?.let { player →
        val mediaItem = MediaItem.fromUri(url)
        player.setMediaItem(mediaItem)
        player.prepare()
        player.play()
    }
}
```

fromUri
Convierte la URL en un elemento reproducible.

setMediaItem
Sustituye el canal actual.

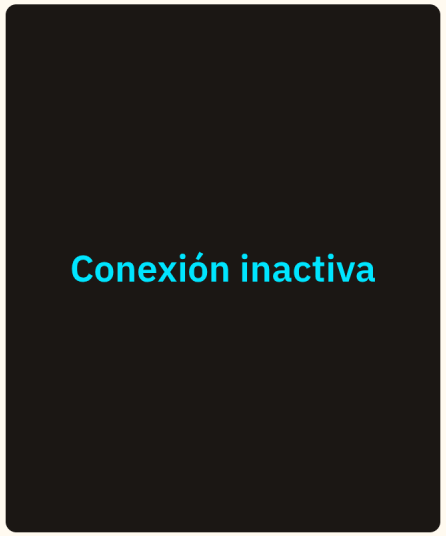
prepare y play
Preparan el búfer y arrancan el vídeo.

Fuente: MainActivity.kt

Los estados del vídeo

El Listener recibe cada cambio en onPlaybackStateChanged y actualiza el mensaje con un when.

STATE_IDLE



Sin contenido preparado para reproducir.

STATE_BUFFERING



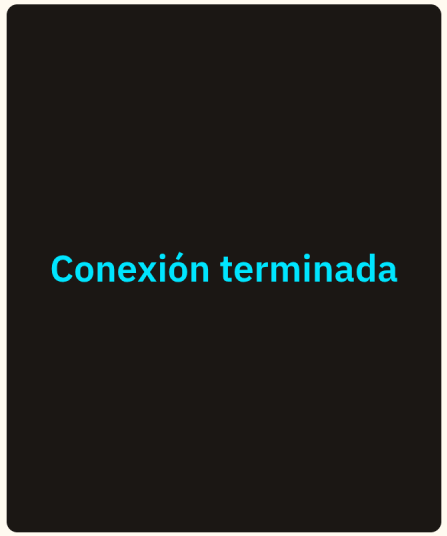
Faltan datos para continuar.

STATE_READY



Listo para reproducir.

STATE_ENDED



Fin del vídeo: suena la alarma.

Fuente: MainActivity.kt

La alarma de campanas

```
private fun reproducirAlarmaAcustica() {
    mediaPlayer?.release()
    mediaPlayer = null
    mediaPlayer = MediaPlayer.create(
        this, R.raw.campanas_alarma
    )?.apply {
        setOnCompletionListener { mp →
            mp.release()
            mediaPlayer = null
        }
        start()
    }
}
```

Limpia
Libera una alarma anterior, si existe.

Reproduce
Crea el MediaPlayer desde res/raw.

Se libera sola
Al acabar, suelta el recurso y queda en null.

Fuente: MainActivity.kt, con saltos de línea para encajar

El ciclo de vida

onStop()

```
exoPlayer?.pause()
```

Si la app pasa a segundo plano, el vídeo se pausa.

onDestroy()

```
exoPlayer?.release()
mediaPlayer?.release()
```

Al destruir la pantalla, se liberan los dos reproductores.

Reproducir bien también es soltar memoria y decodificadores a tiempo.

Las conclusiones

Cada reproductor tiene su trabajo: ExoPlayer para streaming y MediaPlayer para audio local.

El Listener mantiene al usuario informado de lo que ocurre.

Los recursos multimedia deben seguir el ciclo de vida de la Activity.

¿Preguntas?

github.com/antcanor/PMDM